Department of Computerand Software Engineering

Project Proposal

Haseeb Ashfaq **BSCE 23048**

Project Idea: **Spotify Playlist Manager**

1. Introduction

This project aims to create a Spotify-like playlist manager with features for managing music playlists, including options for personalization, playback. By implementing data structures and algorithms, this project will simulate essential Spotify features, providing an interactive and efficient user experience.

2. Project Objective

The objective of this project is to develop a playlist manager that mimics the functionality of Spotify, allowing users to create, customize, and manage playlists with advanced features like song liking, top-played songs, and playback statistics. The project will focus on performance, accuracy, and an intuitive user interface.

3. Key Features

Creating and Managing Playlists

Users can create multiple playlists, name them, and manage their content. Each playlist can hold an array of songs, providing flexibility to organize collections by genre, mood, or preference.

Adding, Removing, and Liking Songs

Users can add or remove songs from any playlist and mark songs as "liked." A "Liked Songs" playlist will automatically store all marked songs, allowing quick access to favorites.

Top 10 Songs

The manager will display the top 10 most-played songs across all playlists. Songs will be ranked based on the number of times they are played, helping users track their favorite music.

Playlist Duration Tracking

Each playlist will calculate and display its total duration, offering users insight into how long the playlist will play before it loops or ends.

Song Search and Filtering

Users can search within playlists or across all playlists, with filtering options by artist, genre, or album. Search functionality will use optimized algorithms for quick retrieval of results.

Shuffle and Play Modes

The playlist manager will include shuffle, repeat, and sequential play modes. Shuffle mode will randomize the song order, while repeat and sequential modes will play the list in user-defined ways.

Sorting

Users can sort playlists based on criteria such as song title, artist, album, or duration. Sorting algorithms will be used to efficiently rearrange playlists.

4. Data Structures and Algorithms Used

Linked List

Dynamic linked lists will manage playlists, allowing flexible addition and removal of songs.

Hash Map for Liked Songs and Top 10

A hash map will store liked songs and keep track of play counts for ranking the top 10 songs.

Sorting Algorithms

Sorting algorithms like QuickSort and MergeSort will organize playlists by various attributes, enhancing navigation and management.

Randomization for Shuffle Feature

A randomization algorithm will implement the shuffle feature, ensuring that each song is played without repetition until all have been played.

5. Challenges and Considerations

Memory and Time Complexity

As playlists grow, the system must manage memory efficiently to handle large data sets and maintain responsive performance.

Consistency of Play Counts

For accurate tracking of top-played songs, the play count needs to be recorded efficiently without interrupting other functionalities.

User-Friendly Interface

While the initial version will use a command-line interface, the data structure and design will support future GUI integration for enhanced user interaction.

6. Future Enhancements

Further features could include a song recommendation system, shared playlists, collaborative editing, and user-generated playlist sharing capabilities.

7. Conclusion

The Spotify Playlist Manager project will simulate key features of a music streaming service, providing a platform for users to manage, personalize, and analyze their music preferences. By combining fundamental DSA concepts with practical application, this project showcases an engaging and real-world use of data structures and algorithms.